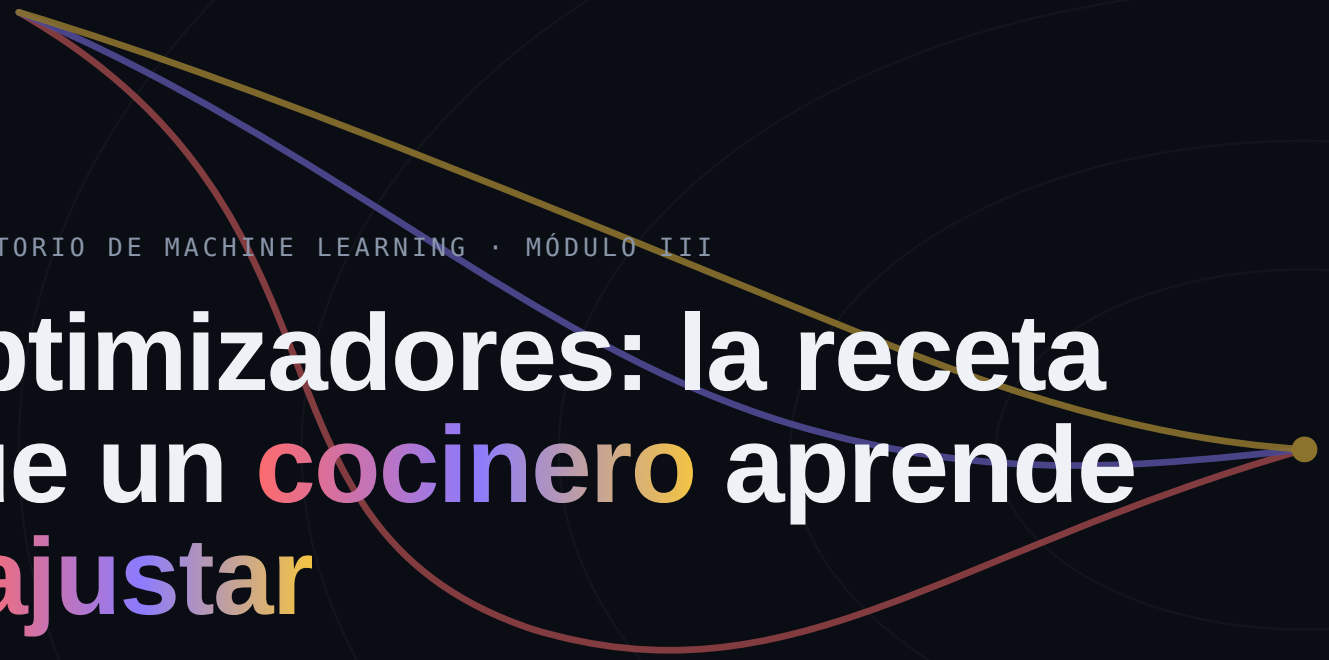


LABORATORIO DE MACHINE LEARNING · MÓDULO III

Optimizadores: la receta que un cocinero aprende a ajustar

Probar, corregir la sazón, volver a probar. Así entrena una red neuronal — y así vamos a entender hoy por qué a veces el plato nunca mejora, y por qué un cocinero con más experiencia siempre le gana al que improvisa a ciegas.

SGD — el aprendiz · Momentum — la mano con memoria · Adam — el chef maestro

Teoría — complemento del notebook interactivo

EL MAPA DE LA ANALOGÍA

Equivalencias clave

Toda la charla de hoy se apoya en una sola idea: **entrenar una red neuronal se parece a ajustar una receta probándola una y otra vez**. Estos son los términos que vamos a usar como sinónimos durante toda la presentación.

🍽️		👅	🥄	🧠	📉
Plato final	**Sazón que se corrige en cada prueba**	**Probar y notar qué le falta**	**Tamaño de la cucharada de ajuste**	**Recordar la tendencia de los últimos ajustes**	**Distancia entre el plato y el sabor ideal**
=	=	=	=	=	=
RED ENTRENADA	PESOS (θ)	GRADIENTE	LEARNING RATE (η)	MOMENTUM	PÉRDIDA (LOSS)

— EL PROBLEMA

El cocinero que decide cuánto y qué corregir

Imaginate una cocinera preparando la misma receta una y otra vez. Cada vez que la prueba, tiene que decidir tres cosas:

- → **Qué corregir** — ¿le falta sal, le sobra ácido, le falta cocción?
- → **Cuánto corregir** — ¿una pizca, o una cucharada entera?
- → **Con memoria o sin memoria** — ¿decide solo por esta probada, o recuerda las últimas correcciones?

LA RECETA GENERAL DE AJUSTE

$$ \theta_{t+1} = \theta_t - \eta \cdot (\text{algo derivado de lo que notamos al probar}) $$

El **optimizador** es exactamente esa decisión, formalizada. Todo lo que distingue a SGD, Momentum y Adam es **qué va dentro de ese paréntesis** — es decir, qué tan sofisticado es el criterio del cocinero para decidir su próximo ajuste.

`η = tamaño de la cucharada · gradiente = lo que notamos al probar`

PUNTO DE PARTIDA

SGD: el cocinero que solo prueba una vez

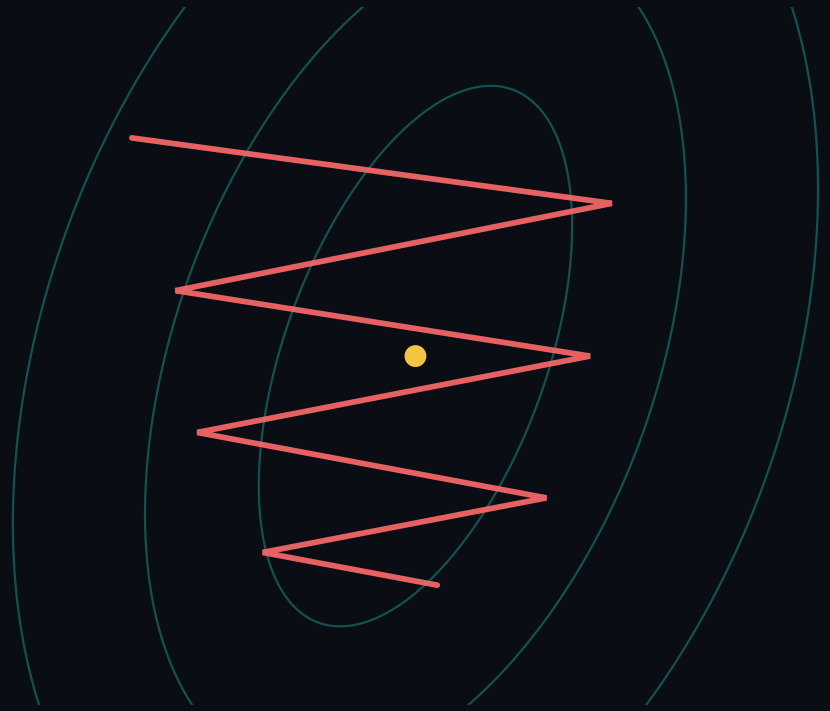
REGLA DE AJUSTE

$$ \theta_{t+1} = \theta_t - \eta \, g_t $$

En cada probada, este cocinero mira **únicamente lo que nota en ese instante** y corrige en consecuencia, con una cucharada de tamaño fijo. No recuerda qué corrigió la vez anterior.

- → Simple y barato: prueba, corrige, listo.
- → Funciona bien cuando el plato es sencillo y el error es parejo.
- × **Es reactivo:** no tiene en cuenta hacia dónde venía corrigiendo.
- × **Una sola cuchara** para todos los ingredientes de la receta, sin importar cuán sensible sea cada uno.

El problema de una sola cuchara para sal y para picante



Pensá en una receta con dos ingredientes muy distintos: **la sal** (con la que hay que ser delicado, se nota con muy poquito) y **el picante** (que tolera cucharadas grandes sin arruinar el plato).

- → **Cuchara grande** → se pasa de sal en cada corrección: **oscila** entre "muy salado" y "muy soso" sin encontrar el punto.
- → **Cuchara chica** → tarda una eternidad en notar diferencia en el picante.

Un mismo tamaño de cuchara no puede ser el correcto para los dos ingredientes al mismo tiempo.

— PRIMERA EVOLUCIÓN

Momentum: la mano que recuerda la tendencia

REGLA DE AJUSTE

$$ v_t = \beta \, v_{t-1} + (1-\beta)\, g_t $$ $$ \theta_{t+1} = \theta_t - \eta \, v_t $$

Idea: en vez de corregir solo por lo que notamos en **esta** probada, promediamos con la **tendencia** de las últimas correcciones.

Como una cocinera con experiencia: si en las últimas tres probadas venía agregando sal de a poquito, su mano ya "sabe" seguir en esa dirección — no reacciona de forma exagerada a un solo bocado raro.

- → Las correcciones que se contradicen entre sí (mucha sal, después mucho ácido, después mucha sal de nuevo) **se cancelan** en el promedio.
- → Las correcciones consistentes (siempre un poquito más de picante) **se refuerzan**.

Resultado: corrige más parejo y llega antes al punto justo, con la misma cuchara que SGD.

— TODAVÍA NO ALCANZA

Una sola cuchara sigue sin ser suficiente

Momentum mejora **hacia dónde** corregimos, pero seguimos usando **la misma cuchara** para todos los ingredientes. En una receta real, cada ingrediente necesita su propio tamaño de ajuste:

- → Ingredientes **fuertes y sensibles** (como la sal) necesitan cucharadas **chicas** — para no arruinar el plato.
- → Ingredientes **suaves o tolerantes** (como el picante) necesitan cucharadas **grandes** — para que la corrección se note.

LA PREGUNTA QUE FALTABA RESPONDER

¿Podemos hacer que **cada ingrediente tenga su propia cuchara** — calibrada automáticamente según cuánto se lo ha tenido que corregir en el pasado?

Esa pregunta es exactamente lo que responden AdaGrad, RMSProp y, finalmente, Adam.

— EL PUENTE

La libreta de anotaciones de cada ingrediente

2011

AdaGrad

$$ \theta_{t+1} = \theta_t - \dfrac{\eta}{\sqrt{\textstyle\sum g_t^2}+\epsilon}\, g_t $$

La cocinera anota en una libreta **cada corrección histórica** de cada ingrediente. Si a la sal la corrigió mucho en el pasado, ahora usa una cuchara más chica para ella — y viceversa.

LIMITACIÓN

La libreta que nunca se borra

$$ \sum g_t^2 \;\longrightarrow\; \infty $$

Como la libreta **solo acumula** y nunca olvida, después de muchas probadas la cuchara se vuelve **microscópica para todo** — la cocinera deja de corregir, aunque el plato todavía lo necesite.

2012

RMSProp

$$ v_t = \beta v_{t-1} + (1-\beta) g_t^2 $$

Cambia la libreta por una **memoria reciente**: solo pesa las últimas correcciones de cada ingrediente. La cuchara se recalibra sola, sin quedarse nunca sin margen de ajuste.

LA SÍNTESIS · 2014

Adam: el chef maestro

1 — RECUERDA LA TENDENCIA (COMO MOMENTUM)

$$ m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t $$

2 — CALIBRA LA CUCHARA DE CADA INGREDIENTE (COMO RMSPROP)

$$ v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 $$

3 — CORRECCIÓN DE ARRANQUE (LAS PRIMERAS PROBADAS CUENTAN POCO)

$$ \hat m_t = \dfrac{m_t}{1-\beta_1^t} \qquad \hat v_t = \dfrac{v_t}{1-\beta_2^t} $$

4 — EL AJUSTE FINAL, INGREDIENTE POR INGREDIENTE

$$ \theta_{t+1} = \theta_t - \dfrac{\eta}{\sqrt{\hat v_t} + \epsilon}\, \hat m_t $$

— POR QUÉ GANÓ

Por qué Adam se volvió el chef de guardia de la industria

- → **Cuchara propia por ingrediente** — cada uno se corrige a su propia medida.
- → **Hereda la memoria de tendencia de Momentum** — no corrige a los tumbos.
- → **No se queda sin margen** como la libreta de AdaGrad.
- → **No hay que adivinar de antemano** el tamaño exacto de cuchara para cada ingrediente.

EN UNA FRASE

Adam le da a cada ingrediente **su propia prudencia**: corrige fuerte donde el plato lo tolera, y con delicadeza donde un exceso arruinaría todo.

Por eso es la cuchara **por defecto** en la enorme mayoría de las cocinas — perdón, de los papers, frameworks y proyectos de deep learning modernos.

— EL OTRO PROBLEMA

La cocina de muchas estaciones: el mensaje que se diluye

Imaginate una cocina con **varias estaciones en cadena**: el caldo base, la reducción, la salsa, el montaje, el emplatado final. El comensal prueba el plato terminado y dice: **"le falta sal"**. Ese mensaje tiene que viajar, estación por estación, de vuelta hasta el caldo base.



En cada estación, el mensaje se repite pero **pierde intensidad** — como el juego del teléfono descompuesto. Para cuando llega al caldo base, el cocinero de esa estación casi no recibe corrección alguna, y sigue preparando el caldo exactamente igual, receta tras receta.

— POR QUÉ PASA ESTO

Por qué el mensaje se diluye tanto

Backpropagation es, literalmente, ese mensaje viajando de estación en estación hacia atrás, **multiplicándose** por cuánto "deja pasar" cada estación:

CORRECCIÓN QUE LLEGA A UNA ESTACIÓN TEMPRANA (L ESTACIONES)

$$ \frac{\partial \mathcal{L}}{\partial \theta_1} \;\propto\; \prod_{l=1}^{L} \sigma'(z_l) $$

Cada estación usa una activación **sigmoide**, cuya derivada tiene un **techo de apenas 0.25** — deja pasar como máximo una cuarta parte del mensaje.

- × Multiplicar 4 o 5 "techos" de 0.25 diluye un mensaje **casi inaudible**.
- × El caldo base (primera capa) casi no recibe corrección.
- × Ese cocinero queda **congelado** en su receta inicial — nunca aprende a mejorar su parte.

— LA SOLUCIÓN PARCIAL

Cómo Adam ayuda (pero no arregla la cadena de estaciones)

El ajuste de Adam se recalibra dividiendo por $\sqrt{\hat{v}_t}$ — la magnitud típica reciente de la corrección **de ese ingrediente puntual**:

$$ \Delta\theta \;=\; \dfrac{\hat m_t}{\sqrt{\hat v_t} + \epsilon} $$

Si el caldo base viene recibiendo correcciones consistentemente chicas, Adam **agranda su cuchara** para compensar — en vez de dejar que la corrección colapse a casi nada, como le pasaría a SGD.

IMPORTANTE

Adam **no achica la cadena de estaciones**: el mensaje real que llega al caldo base sigue siendo débil, Adam solo lo aprovecha mejor.

Por eso, en la práctica moderna, se combina Adam con arreglos a la cocina misma:

- → **ReLU** en vez de sigmoide (deja pasar el mensaje sin techo de 0.25).
- → **Buena "puesta a punto" inicial** de cada estación (inicialización He / Xavier).
- → **Estandarizar** las porciones en cada estación (Batch / Layer Normalization).
- → **Atajos directos** entre estaciones lejanas (conexiones residuales).

RESUMEN

Tres cocineros, tres formas de corregir la receta

COCINERO	RECUERDA LA TENDENCIA	CUCHARA PROPIA POR INGREDIENTE	RIESGO DE ARRUINAR EL PLATO	FRENTE AL MENSAJE DILUIDO
SGD	No	No	Alto	Sin mitigación
Momentum	Sí	No	Medio	Mitigación leve
Adam	Sí	**Sí**	**Bajo**	Mitigación notable

— EN LA PRÁCTICA

De la cocina al laboratorio: esto lo probamos con datos reales

- → Trayectorias de SGD / Momentum / Adam sobre un valle angosto — la versión matemática de "la sal y el picante".
- → Una red neuronal **desde cero, con NumPy**, entrenada sobre el dataset público *Iris* (flores — versicolor vs. virginica).
- → La norma del gradiente **por capa**, capturada en vivo — la versión medible de "cuánto le llega el mensaje a cada estación".

LO QUE SE COMPROBÓ EMPÍRICAMENTE

Con **SGD** puro, la primera capa (el "caldo base") quedó recibiendo un mensaje **órdenes de magnitud** más débil que la última. Con **Adam**, esa brecha se achicó y la red aprendió más rápido y de forma más estable.

GRACIAS

De un mensaje que se diluye a un cocinero que calibra cada cuchara

Preguntas, dudas, o ganas de probar la receta ustedes mismos en el notebook interactivo — todo queda abierto a discusión.